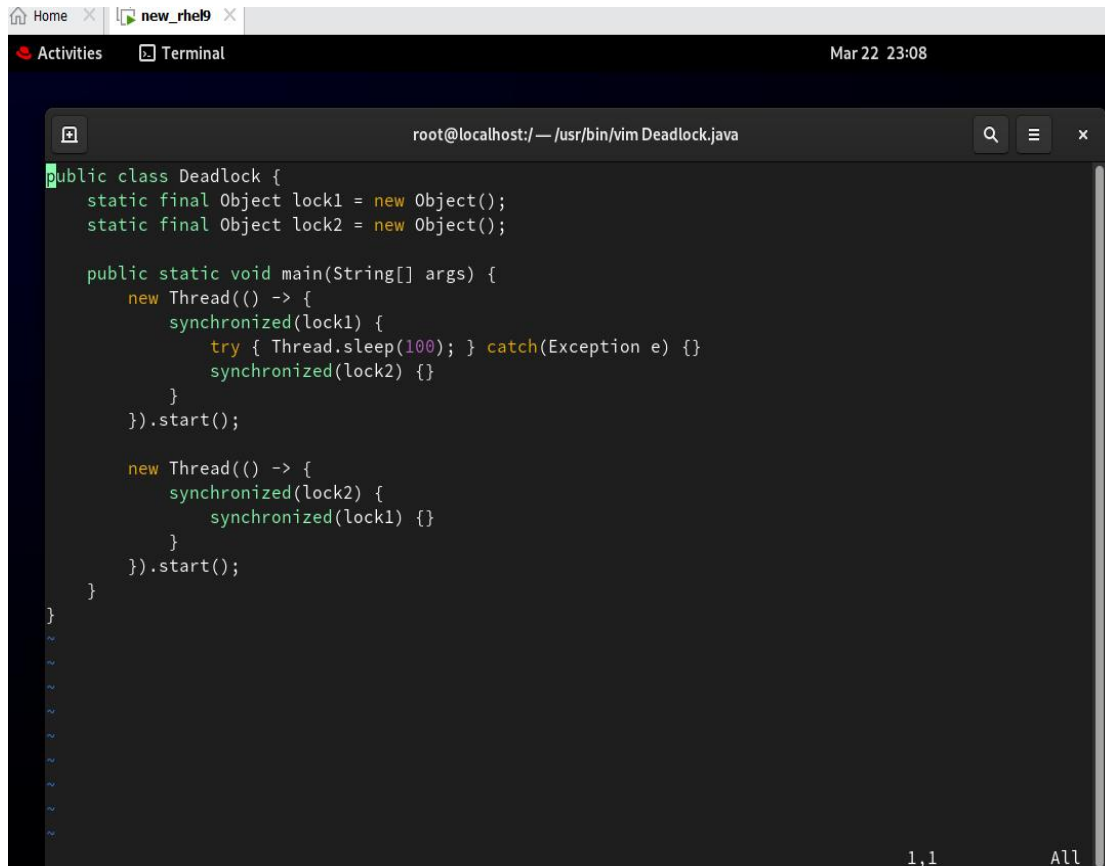


Step1. I created a simple Java program/process (Deadlock.java) just for practice, so I can learn how to take a Java thread dump.



```
root@localhost: /usr/bin/vim Deadlock.java
public class Deadlock {
    static final Object lock1 = new Object();
    static final Object lock2 = new Object();

    public static void main(String[] args) {
        new Thread() -> {
            synchronized(lock1) {
                try { Thread.sleep(100); } catch(Exception e) {}
                synchronized(lock2) {}
            }
        }.start();

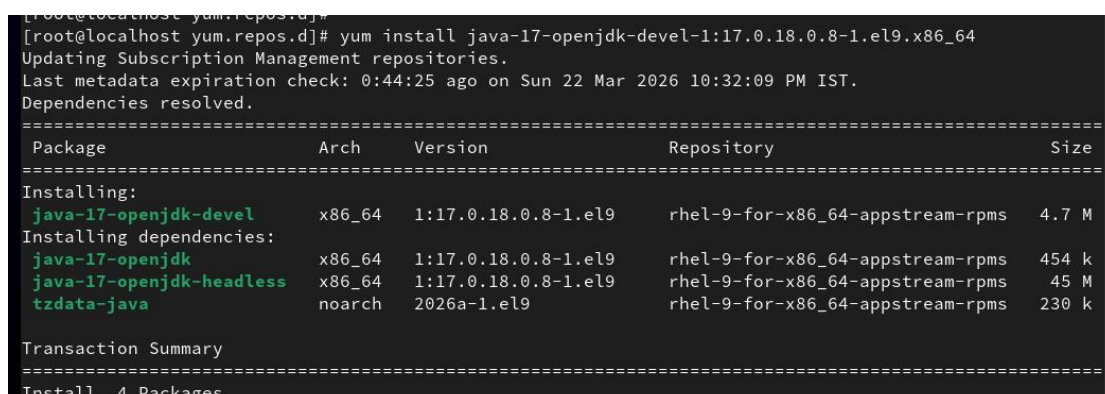
        new Thread() -> {
            synchronized(lock2) {
                synchronized(lock1) {}
            }
        }.start();
    }
}
```

step2. First, I verified that the Java file (Deadlock.java) is created in the system.



```
[root@localhost /]# ls
afs boot Deadlock.java etc lib media opt root sbin sys usr vardirbckp
```

step3. To run a Java program, Java (JDK) must be installed on the server. So, I installed Java using the package manager (yum install java-17-openjdk-devel). now u can run comand like javac,jstack,java,jps.



```
[root@localhost yum.repos.d]# yum install java-17-openjdk-devel-1:17.0.18.0.8-1.el9.x86_64
Updating Subscription Management repositories.
Last metadata expiration check: 0:44:25 ago on Sun 22 Mar 2026 10:32:09 PM IST.
Dependencies resolved.
=====
Package                                Arch      Version              Repository            Size
=====
Installing:
java-17-openjdk-devel                  x86_64    1:17.0.18.0.8-1.el9  rhel-9-for-x86_64-appstream-rpms  4.7 M
Installing dependencies:
java-17-openjdk                        x86_64    1:17.0.18.0.8-1.el9  rhel-9-for-x86_64-appstream-rpms   454 k
java-17-openjdk-headless               x86_64    1:17.0.18.0.8-1.el9  rhel-9-for-x86_64-appstream-rpms   45 M
tzdata-java                           noarch    2026a-1.el9          rhel-9-for-x86_64-appstream-rpms   230 k
Transaction Summary
=====
Install 4 Packages
```

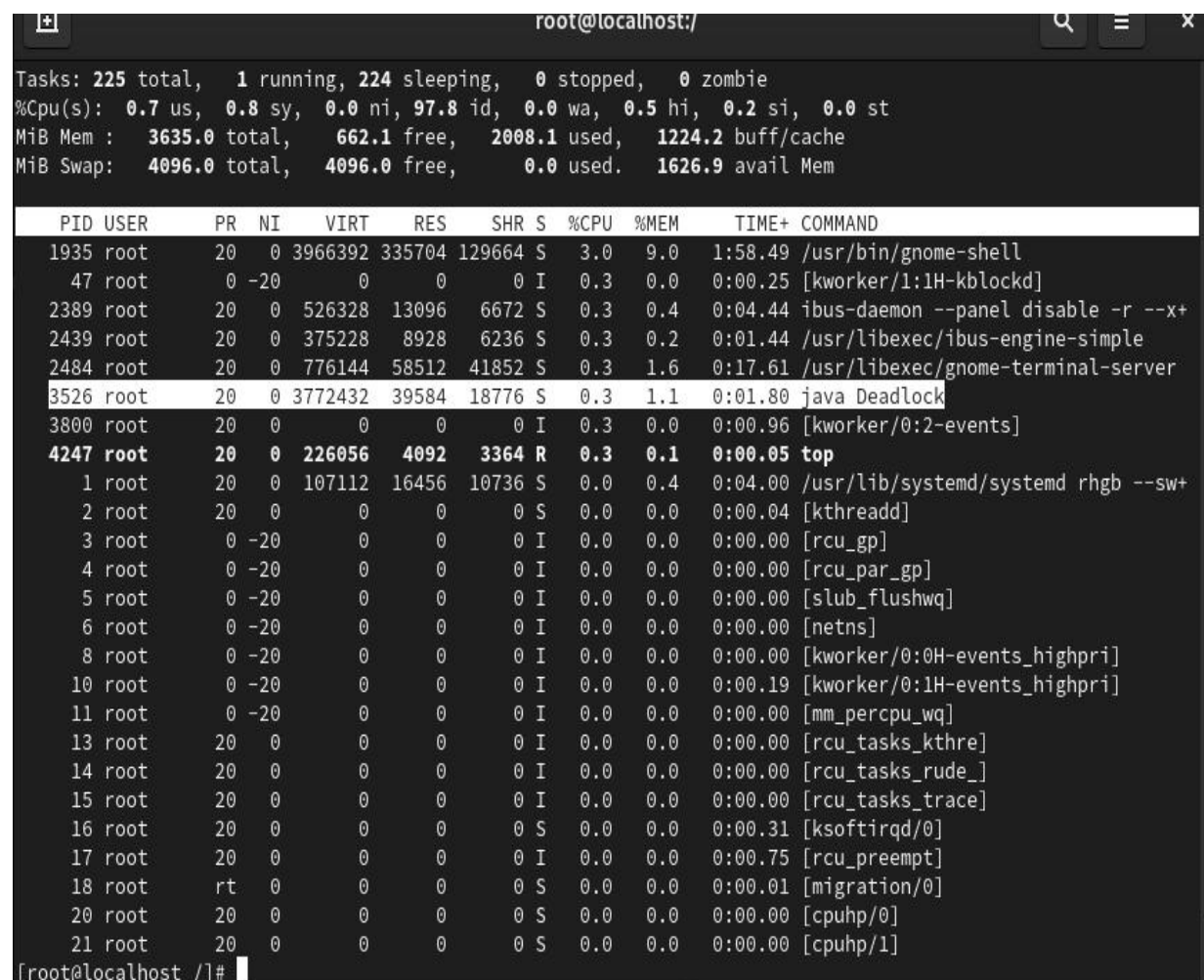
Step4. After installing Java, I compiled the program using: **javac Deadlock.java** this command creates a bytecode file (Deadlock.class).

Then, I ran the program using: **java Deadlock** This starts the Java application.

```
[root@localhost ~]#  
[root@localhost ~]# javac Deadlock.java  
[root@localhost ~]# java Deadlock
```

Step5. Verify Application is Running and Find PID

After running the Java program, I verified whether the application is running. First, I used the **top** command to check running processes and confirmed that the **java Deadlock** process is active.



Tasks: 225 total, 1 running, 224 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.7 us, 0.8 sy, 0.0 ni, 97.8 id, 0.0 wa, 0.5 hi, 0.2 si, 0.0 st
MiB Mem : 3635.0 total, 662.1 free, 2008.1 used, 1224.2 buff/cache
MiB Swap: 4096.0 total, 4096.0 free, 0.0 used, 1626.9 avail Mem

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1935	root	20	0	3966392	335704	129664	S	3.0	9.0	1:58.49	/usr/bin/gnome-shell
47	root	0	-20	0	0	0	I	0.3	0.0	0:00.25	[kworker/1:1H-kblockd]
2389	root	20	0	526328	13096	6672	S	0.3	0.4	0:04.44	ibus-daemon --panel disable -r --x+
2439	root	20	0	375228	8928	6236	S	0.3	0.2	0:01.44	/usr/libexec/ibus-engine-simple
2484	root	20	0	776144	58512	41852	S	0.3	1.6	0:17.61	/usr/libexec/gnome-terminal-server
3526	root	20	0	3772432	39584	18776	S	0.3	1.1	0:01.80	java Deadlock
3800	root	20	0	0	0	0	I	0.3	0.0	0:00.96	[kworker/0:2-events]
4247	root	20	0	226056	4092	3364	R	0.3	0.1	0:00.05	top
1	root	20	0	107112	16456	10736	S	0.0	0.4	0:04.00	/usr/lib/systemd/systemd rhgb --sw+
2	root	20	0	0	0	0	S	0.0	0.0	0:00.04	[kthreadd]
3	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	[rcu_gp]
4	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	[rcu_par_gp]
5	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	[slub_flushwq]
6	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	[netns]
8	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	[kworker/0:0H-events_highpri]
10	root	0	-20	0	0	0	I	0.0	0.0	0:00.19	[kworker/0:1H-events_highpri]
11	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	[mm_percpu_wq]
13	root	20	0	0	0	0	I	0.0	0.0	0:00.00	[rcu_tasks_kthre]
14	root	20	0	0	0	0	I	0.0	0.0	0:00.00	[rcu_tasks_rude_]
15	root	20	0	0	0	0	I	0.0	0.0	0:00.00	[rcu_tasks_trace]
16	root	20	0	0	0	0	S	0.0	0.0	0:00.31	[ksoftirqd/0]
17	root	20	0	0	0	0	I	0.0	0.0	0:00.75	[rcu_preempt]
18	root	rt	0	0	0	0	S	0.0	0.0	0:00.01	[migration/0]
20	root	20	0	0	0	0	S	0.0	0.0	0:00.00	[cpuhp/0]
21	root	20	0	0	0	0	S	0.0	0.0	0:00.00	[cpuhp/1]

[root@localhost ~]#

Step 6. Then, I used the `jps -l` command to list Java processes, where I could see the `Deadlock` program along with its PID. I also used the `ps -ef | grep Deadlock` command to find the process manually. Additionally, I verified using `ps -ef | grep java` to check all Java-related processes.

```
[root@localhost ~]#  
[root@localhost ~]#  
[root@localhost ~]# jps -l  
4114 jdk.jcmd/sun.tools.jps.Jps  
3526 Deadlock  
[root@localhost ~]#  
[root@localhost ~]#  
[root@localhost ~]#  
[root@localhost ~]# ps -ef | grep Deadlock  
root      3526      2503  0 23:37 pts/0    00:00:01 java Deadlock  
root      4158      3547  0 23:52 pts/1    00:00:00 grep --color=auto Deadlock  
[root@localhost ~]#  
[root@localhost ~]#  
[root@localhost ~]# ps -ef | grep java  
root      3526      2503  0 23:37 pts/0    00:00:01 java Deadlock  
root      4172      3547  0 23:52 pts/1    00:00:00 grep --color=auto java  
[root@localhost ~]#  
[root@localhost ~]#  
[root@localhost ~]#
```

Step7. The Java program is running, but after some time it gets stuck and stops working. Even though the process is active, it is not performing any operation. This indicates there may be an issue,

So I decided to take a thread dump to analyze the problem.

```
[root@localhost ~]# java Deadlock
```

step8. Before taking the thread dump, I checked the TIDs (Thread IDs) of the Java process using its PID by running the command `top -H -p <PID>`

I observed that none of the threads were consuming high CPU or memory. Even though resource usage was normal, the application was still stuck. This indicates that the issue is not related to CPU or memory, but possibly due to a deadlock or thread waiting condition.

```
[root@localhost ~]# top -H -p 3526

top - 23:58:53 up 41 min, 2 users, load average: 1.14, 0.67, 0.43
Threads: 21 total, 0 running, 21 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.3 us, 0.3 sy, 0.0 ni, 98.8 id, 0.0 wa, 0.3 hi, 0.2 si, 0.0 st
MiB Mem : 3635.0 total, 652.8 free, 2017.3 used, 1224.2 buff/cache
MiB Swap: 4096.0 total, 4096.0 free, 0.0 used, 1617.7 avail Mem

  PID USER      PR  NI   VIRT   RES   SHR S  %CPU  %MEM    TIME+  COMMAND
 3526 root        20   0 3772432 39672 18776 S   0.0   1.1   0:00.00 java
 3527 root        20   0 3772432 39672 18776 S   0.0   1.1   0:00.06 java
 3528 root        20   0 3772432 39672 18776 S   0.0   1.1   0:00.00 GC Thread#0
 3529 root        20   0 3772432 39672 18776 S   0.0   1.1   0:00.00 G1 Main Marker
 3530 root        20   0 3772432 39672 18776 S   0.0   1.1   0:00.00 G1 Conc#0
 3531 root        20   0 3772432 39672 18776 S   0.0   1.1   0:00.00 G1 Refine#0
 3532 root        20   0 3772432 39672 18776 S   0.0   1.1   0:00.31 G1 Service
 3533 root        20   0 3772432 39672 18776 S   0.0   1.1   0:00.05 VM Thread
 3534 root        20   0 3772432 39672 18776 S   0.0   1.1   0:00.00 Reference Handl
 3535 root        20   0 3772432 39672 18776 S   0.0   1.1   0:00.00 Finalizer
 3536 root        20   0 3772432 39672 18776 S   0.0   1.1   0:00.00 Signal Dispatch
 3537 root        20   0 3772432 39672 18776 S   0.0   1.1   0:00.00 Service Thread
 3538 root        20   0 3772432 39672 18776 S   0.0   1.1   0:00.21 Monitor Deflati
 3539 root        20   0 3772432 39672 18776 S   0.0   1.1   0:00.01 C2 CompilerThre
 3540 root        20   0 3772432 39672 18776 S   0.0   1.1   0:00.00 C1 CompilerThre
 3541 root        20   0 3772432 39672 18776 S   0.0   1.1   0:00.00 Sweeper thread
 3542 root        20   0 3772432 39672 18776 S   0.0   1.1   0:00.00 Notification Th
 3543 root        20   0 3772432 39672 18776 S   0.0   1.1   0:01.29 VM Periodic Tas
 3544 root        20   0 3772432 39672 18776 S   0.0   1.1   0:00.00 Common-Cleaner
 3545 root        20   0 3772432 39672 18776 S   0.0   1.1   0:00.05 Thread-0
 3546 root        20   0 3772432 39672 18776 S   0.0   1.1   0:00.04 Thread-1
```

step9. so, I took the thread dump using the `jstack` command with the PID of the Java process. After running the command, I verified that the thread dump file was successfully created.

This dump file will be used to analyze the issue in the Java application.

```
root@localhost:/

[root@localhost ~]# jstack 3526 > /tmp/threaddump.java
[root@localhost ~]#
[root@localhost ~]#
[root@localhost ~]# ls /tmp
hsperfdata_root  threaddump.java
[root@localhost ~]#
[root@localhost ~]#
[root@localhost ~]#
[root@localhost ~]#
[root@localhost ~]#
```

step10. Analyze Thread Dump

After taking the thread dump, I opened the dump file to analyze it. The thread dump shows the state of all threads in the Java application.

In the dump file, I can see that some threads are in a waiting/blocked state. It clearly shows that one thread is waiting to acquire a lock held by another thread, and the second thread is waiting for the first thread.

The dump also explicitly mentions "Found one Java-level deadlock", which confirms the issue. It provides details of both threads, including which locks they are holding and which locks they are waiting for.

From this thread dump, I confirmed that the application is stuck due to a deadlock.

```
root@localhost:/tmp — /usr/bin/vim threaddump.java

"G1 Conc#0" os_prio=0 cpu=0.11ms elapsed=1453.57s tid=0x00007f85b406c740 nid=0xdca runnable
"G1 Main Marker" os_prio=0 cpu=0.16ms elapsed=1453.57s tid=0x00007f85b406b7d0 nid=0xdc9 runnable
"GC Thread#0" os_prio=0 cpu=0.12ms elapsed=1453.57s tid=0x00007f85b4063030 nid=0xdc8 runnable

JNI global refs: 4, weak refs: 0

Found one Java-level deadlock:
=====
"Thread-0":
  waiting to lock monitor 0x00007f850c046840 (object 0x00000000cab19488, a java.lang.Object),
  which is held by "Thread-1"

"Thread-1":
  waiting to lock monitor 0x00007f8518001330 (object 0x00000000cab19478, a java.lang.Object),
  which is held by "Thread-0"

Java stack information for the threads listed above:
=====
"Thread-0":
  at Deadlock.lambda$main$0(Deadlock.java:9)
  - waiting to lock <0x00000000cab19488> (a java.lang.Object)
  - locked <0x00000000cab19478> (a java.lang.Object)
  at Deadlock$$Lambda$1/0x00007f8544000a08.run(Unknown Source)
  at java.lang.Thread.run(java.base@17.0.18/Thread.java:840)
"Thread-1":
  at Deadlock.lambda$main$1(Deadlock.java:15)
  - waiting to lock <0x00000000cab19478> (a java.lang.Object)
  - locked <0x00000000cab19488> (a java.lang.Object)
  at Deadlock$$Lambda$2/0x00007f8544000c28.run(Unknown Source)
  at java.lang.Thread.run(java.base@17.0.18/Thread.java:840)

Found 1 deadlock.
```


step11 . After analyzing the thread dump file, I identified the problematic threads using their nid (native thread ID).The thread dump shows nid in hexadecimal format (e.g., 0xdd9, 0xdda).

To match these with system-level thread IDs (TIDs), I converted the nid values from hexadecimal to decimal using the `printf` command.

```
Thread-0" i12 prio=5 os_prio=0 cpu=62.63ms elapsed=1453.47s tid=0x00007f85b40f7070 nid=0xdd9 waiting for monitor entry [0x00007f85993f6000]
java.lang.Thread.State: BLOCKED (on object monitor)
    at Deadlock.lambda$main$0(Deadlock.java:9)
    - waiting to lock <0x00000000cab19488> (a java.lang.Object)
    - locked <0x00000000cab19478> (a java.lang.Object)
    at Deadlock$$Lambda$1/0x00007f8544000a08.run(Unknown Source)
    at java.lang.Thread.run(java.base@17.0.18/Thread.java:840)

Thread-1" i13 prio=5 os_prio=0 cpu=59.73ms elapsed=1453.47s tid=0x00007f85b40f8300 nid=0xdda waiting for monitor entry [0x00007f85992f5000]
java.lang.Thread.State: BLOCKED (on object monitor)
    at Deadlock.lambda$main$1(Deadlock.java:15)
    - waiting to lock <0x00000000cab19478> (a java.lang.Object)
    - locked <0x00000000cab19488> (a java.lang.Object)
    at Deadlock$$Lambda$2/0x00007f8544000c28.run(Unknown Source)
    at java.lang.Thread.run(java.base@17.0.18/Thread.java:840)
```

```
[root@localhost tmp]# printf "%d\n" 0xdd9
3545
[root@localhost tmp]#
[root@localhost tmp]#
[root@localhost tmp]#
[root@localhost tmp]# printf "%d\n" 0xdda
3546
```

step12 .After converting the nid values to TIDs, I matched them with the threads using the `top -H -p 3526` command.I identified the problematic threads in the system.Both threads were in a waiting state and not consuming CPU.From the analysis, it is clear that both threads are waiting for each other's resources.

Threads: 22 total, 0 running, 22 sleeping, 0 stopped, 0 zombie											
Cpu(s): 0.0 us, 0.3 sy, 0.0 ni, 99.0 id, 0.0 wa, 0.5 hi, 0.2 si, 0.0 st											
Mem: 3635.0 total, 584.5 free, 2088.6 used, 1238.8 buff/cache											
Mem Swap: 4096.0 total, 4096.0 free, 0.0 used, 1546.4 avail Mem											
PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
3543	root	20	0	3773460	39760	18776	S	0.3	1.1	0:02.08	VM Peri+
3526	root	20	0	3773460	39760	18776	S	0.0	1.1	0:00.00	java
3527	root	20	0	3773460	39760	18776	S	0.0	1.1	0:00.06	java
3528	root	20	0	3773460	39760	18776	S	0.0	1.1	0:00.00	GC Thre+
3529	root	20	0	3773460	39760	18776	S	0.0	1.1	0:00.00	G1 Main+
3530	root	20	0	3773460	39760	18776	S	0.0	1.1	0:00.00	G1 Conc+
3531	root	20	0	3773460	39760	18776	S	0.0	1.1	0:00.00	G1 Refi+
3532	root	20	0	3773460	39760	18776	S	0.0	1.1	0:00.49	G1 Serv+
3533	root	20	0	3773460	39760	18776	S	0.0	1.1	0:00.08	VM Thre+
3534	root	20	0	3773460	39760	18776	S	0.0	1.1	0:00.00	Referen+
3535	root	20	0	3773460	39760	18776	S	0.0	1.1	0:00.00	Finaliz+
3536	root	20	0	3773460	39760	18776	S	0.0	1.1	0:00.00	Signal +
3537	root	20	0	3773460	39760	18776	S	0.0	1.1	0:00.00	Service+
3538	root	20	0	3773460	39760	18776	S	0.0	1.1	0:00.35	Monitor+
3539	root	20	0	3773460	39760	18776	S	0.0	1.1	0:00.02	C2 Comp+
3540	root	20	0	3773460	39760	18776	S	0.0	1.1	0:00.01	C1 Comp+
3541	root	20	0	3773460	39760	18776	S	0.0	1.1	0:00.00	Sweeper+
3542	root	20	0	3773460	39760	18776	S	0.0	1.1	0:00.00	Notific+
3544	root	20	0	3773460	39760	18776	S	0.0	1.1	0:00.00	Common+
3545	root	20	0	3773460	39760	18776	S	0.0	1.1	0:00.07	Thread-0
3546	root	20	0	3773460	39760	18776	S	0.0	1.1	0:00.08	Thread-1